UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Universitatea Tehnică din Cluj-Napoca

Set de programe benchmark pentru a evalua performanța unui procesor multicore

Proiect Structura Sistemelor de Calcul

Student: Blaga Alexandra
Facultatea de Automatică și Calculatoare
Gr. 30234
Profesor proiect: Bozdog Raluca - Delia
2025-2026

Cuprins

1. Introducere 4
2. Studiu Bibliografic 4
2.1. Benchmarking 4
2.2. Procesoare Multicore și Paralelismul 4
2.3. Tehnici de măsurare 5
2.4. Algoritmi utilizați în Benchmarking 5
2.5. Surse de eroare și variații ale performanței 5
3. Plan de lucru 5
4. Analiză 6
4.1 Informațiile obținute cu ajutorul instrucțiunii CPUID 6
4.2 Măsurarea timpului de execuție folosind controlul de cicluri al procesorului (instrucțiunile RDTSC și CPUID) 8
4.3 Metoda Monte Carlo Pentru aproximarea lui π 8
4.4 Generator de numere pseudo-aleatoare 9
4.5. Înmulțirea matricelor 9
4.6. Transformata Fourier Rapidă 10
4.7. Integrarea într-o bibliotecă DLL 10
5. Design 10
5.1. Structura generală și organizarea pe module C++ 10
5.2. DLL 12
5.3. Interfața cu utilizatorul: C# + WinForms 12
6. Implementare 13
6.1. Detecția hardware (GetCpuInfo) 13
6.2. Benchmark Monte Carlo 14
6.3. Benchmark-ul de înmulțire a matricilor 15
6.4. Benchmark-ul FFT (Fast Fourier Transform) 16
6.5. Impelentarea DLL-ului 18
6.6. Implementarea interoperabilității cu DLL în C# 18
6.7. Implementarea interfeței grafice în C# 19
7. Aplicația 20
7.1. Zona de afișare a informațiilor despre procesor 20
7.2. Configurarea Benchmark-urilor 21

7.3. Zona de rezultate 24
7.4. Modul de utilizare al aplicației 26
8. Testare 26
8.1. Testarea pe sistemul propriu 27
8.2. Testarea pe sisteme suplimentare 28
8.3. Metodologia de notare și sistemul de punctaj 30
9. Concluzii 31
9.1. Probleme întâmpinate 31
9.2. Posibile îmbunătățiri 32
9.3. Concluzie 32
Bibliografie 32

1. Introducere

Procesoarele moderne dispun de nuclee multiple pentru a executa sarcini în paralel. Totuși, câștigul real de performanță rezultat prin utilizarea multithreading-ului depinde de tipul algoritmului, de sincronizarea efectuată și de metodele de accesare a memoriei.

Acest proiect are ca scop dezvoltarea unei aplicații desktop capabilă de a evalua performanța procesoarelor cu nuclee multiple, utilizând diferite sarcini de lucru, precum simularea Monte Carlo, înmulțirea matricelor și Transformata Fourier Rapidă. Aplicația de testare (benchmark) va fi implementată în C++ și C#, nucleul acesteia fiind realizat în C++, iar interfața grafică pentru configurarea testelor și vizualizarea rezultatelor va fi creată în C#.

Programul va afișa informații despre procesor, obținute prin intermediul CPUID și îi va permite utilizatorului să aleagă algoritmul, să configureze numărul de fire de execuție (threads) și să vizualizeze metrici de performanță precum și grafice sugestive.

2. Studiu Bibliografic

2.1. Benchmarking

Benchmarking-ul reprezintă o abordare structurată pentru a evalua performanța sistemelor informatice în condiții controlate. Conform lui Bouckaert și colab. [1], un benchmark bun trebuie să fie repetabil, reprezentativ și independent de detaliile specifice hardware-ului sau de implementare, asigurând o comparație corectă între diferite sisteme și configurații. Aceste principii se aplică benchmark-urilor de rețea și de evaluare a performanței procesorului.

În cazul procesoarelor conservarea condițiilor de testare și a indicatorilor, sunt esențiale pentru rezultate semnificative. Benchmark-urile evaluează viteza de execuție pentru un anumit set de sarcini, printre care operații aritmetice sau algoritmi paralelizați. Acestea pot fi de mai multe tipuri: sintetice, care utilizează teste simplificate (ex. CPU-Z, Geekbench), application-based, care se bazează pe programe reale (ex Cinebench, SPEC CPU [2]), componente, care sunt utilizate pentru a determina parametri hardware, kernel sau paralele, care sunt utilizate pentru sisteme cu nuclee multiple, procesoare multiple sau sisteme care consistă din mai multe calculatoare.

Scopul acestui proiect este de a implementa un benchmark personalizat capabil să testeze performanța multicore, folosind sarcini de lucru și număr de thread-uri configurabile, concentrând-se pe numărul de cicluri necesare execuției ca metrică principală.

2.2. Procesoare Multicore și Paralelismul

Procesoarele moderne conțin multiple nuclee care pot executa mai multe instrucțiuni simultan (în paralel). Execuția paralelă îmbunătățește performanța, dar aceasta introduce și costuri suplimentare de sincronizare si comunicare.

Conform legii lui Amdahl [3], creșterea maximă a vitezei care poate fi obținută prin paralelizare depinde de porțiunea din program care poate fi executată în paralel. În practică, alți factori precum

utilizarea cache-ului, planificarea firelor de execuție și lățimea de bandă a memoriei limitează scalabilitatea (capacitate de extindere a performanței). Înțelegerea acestor efecte este esențială atunci când se interpretează rezultatele timpului de execuție obținute din teste multithread.

2.3. Tehnici de măsurare

Timpul de execuție este un indicator direct al performanței. Acesta reprezintă timpul total necesar pentru a finaliza un calcul și include partea de calculare, sincronizare și întârzierile de sistem. Se pot utiliza mai multe metode pentru aflarea timpului de execuție printre care:

- Temporizatoare de nivel înalt – **std::chrono**[4], portabil și simplu
- Temporizatoare de sistem – **QueryPerformanceCounter()**[5], exact, dar specific pentru Windows
- Temporizatoare hardware – **RDTSC** (read time-stamp counter)[6], precis, temporizarea fiind la nivel de ciclu, însă necesită forțarea execuției ordonate (serializarea) prin instrucțiunea CPUID [7].

2.4. Algoritmi utilizați în Benchmarking

Proiectul utilizează trei sarcini de lucru computaționale bine cunoscute pentru a evalua performanța procesoarelor multicore:

- **Estimarea lui π prin metoda Monte Carlo** [8] este un algoritm intens dependent de calcule, complet paralelizabil, care realizează calcule aleatorii independente pentru a aproxima valoarea lui π. Fiecare fir de execuție poate funcționa independent, ceea ce îl face ideal pentru evaluarea paralelismului brut al procesorului, cu o suprasarcină minima de sincronizare.
- **Înmulțirea matricelor reprezintă** [9] o sarcină mixtă, dependentă atât de calcule, cât și de memorie, implicând bucle imbricate și volume mari de date. Acestea solicită atât debitul aritmetic, cât și lățimea de bandă a memorie.
- **Transformata Fourier Rapidă** [10] este un algoritm pentru semnalelor digitale, care efectuează numeroase operații aritmetice în virgulă mobilă. Algoritmul reprezintă o sarcină mixtă, dependentă de calcule și de memorie, fiind adesea utilizat pentru testarea performanței arhitecturilor moderne în aplicații numerice complexe.

2.5. Surse de eroare și variații ale performanței

O evaluare precisă trebuie să ia în considerare și factorii de mediu și arhitecturali care induc zgomot sau variabilitate precum: limitarea termică, scalarea dinamică a frecvenței, migrarea firelor de execuție, multithreading simultan, păstrarea locației în memorie.

3. Plan de lucru

1. Săp. 1: Alegerea proiectului.
2. Săp. 3: Introducere, studiu bibliografic, plan de lucru.
3. Săp. 5: Analiză, design inițial al UI, implementare Monte Carlo.
4. Săp. 7: Implementare înmulțirea matricelor, transformata Fourier rapidă, îmbunătățire UI.
5. Săp. 9: Finalizare UI, generare grafice.

6. Săp. 11: Testare, finalizare documentație.

4. Analiză

Aplicația își propune să ofere utilizatorului atât posibilitatea de a vizualiza informații despre procesorul care urmează a fi testat, cât și testarea acestuia prin metodologii diferite și în condiții diferite. Toate acestea sunt prezentate în cele ce urmează:

4.1 Informațiile obținute cu ajutorul instrucțiunii CPUID

Funcția `GetCpuInfo(CpuInfo* Info)` extrage informații despre procesor (vendor, nume comercial, număr de nuclee și thread-uri, dimensiunea memoriilor cache) folosind instrucțiunea CPUID apelată prin cod în limbaj de asamblare. Pașii generali pentru fiecare apel CPUID sunt următorii:

1. Se pregătesc registrele de intrare:
    - `EAX` primește codul pentru informațiile care doresc a fi extrase
    - Uneori `ECX` primește un subcod, fiind o valoare suplimentară de selectare a variantei de informații
2. Se execută instrucțiunea `CPUID`
3. Se salvează conținutul registrelor `EAX`, `EBX`, `ECX`, `EDX` la adresele unor variabile locale
4. Se decodează biții din aceste registre conform convențiilor Intel/AMD

Funcția colectează și interpretează următoarele categorii de date:

a) Vendor string / identificatorul producătorului (CPUID cu EAX = 0x00000000) [11]

Apelul `CPUID` cu registrul `EAX` având valoarea 0 returnează în ordine în registrele `EBX`, `EDX`, și `ECX` un șir de numere care reprezintă codificarea unor caractere în codul ASCII. Interpretarea codului rezultă într-un string specific fiecărui producător de procesoare, precum:

- „GenuineIntel” – Intel
- ”AuthenticAMD” – AMD

Acest câmp este ulterior folosit ca să alegem căile de detecție specifice Intel sau AMD (ex. pentru numărul de nuclee fizice).

b) Brand string / denumirea comercială a procesorului (CPUID cu EAX = 0x80000002, 0x80000003, 0x80000004)

Denumirea comercială a procesorului conține informații cum ar fi producătorul, tipul procesorului, generația si frecvența de lucru pentru unele modele. În cazul PC-ului pe care am rulat primele teste, frecvența de bază și cea maximă nu pot fi accesat în niciun mod prin intermediul instrucțiunii CPUID, însă string-ul returnat este:

AMD Ryzen 9 4900HS with Radeon Graphics

Pentru aflarea denumirii comerciale a procesorului este nevoie de 3 apeluri succesive ale instrucțiunii `CPUID` pentru a obține întregul șir de caractere. Fiecare registru are la dispoziție 32 de biți, 4 octeți, ceea

ce înseamnă posibilitatea de a reține 4 caractere. Un apel `CPUID` generează 4 registre, adică 16 caractere, iar pentru redarea oricărui string de mai mult de 32 de caractere este nevoie de 3 apeluri ale instrucțiunii.

c) Numărul de thread-uri logice (CPUID cu EAX = 0x00000001)

Instrucțiunea `CPUID` apelată cu valoarea 1 în registrul EAX returnează informații legate de topologia logică a procesorului. În special, câmpul `EBX [23:16]` conține numărul maxim de procesoare logice (thread-uri hardware) care pot fi adresate în cadrul aceluiași pachet fizic. Valoarea obținută este decodificată prin deplasarea la dreapta a registrului `EBX` cu 16 biți și mascarea cu `0xFF`. Aceasta reprezintă numărul de thread-uri hardware disponibile, incluzând cele rezultate prin tehnologii de tip Hyper-Threading / SMT.

De asemenea, pentru a verifica dacă procesorul suportă Hyper-Threading, se poate folosi bitul `EDX[28]` din același apel `CPUID(1)`, care indică suportul pentru HTT (Hyper-Threading Technology).

d) Numărul de nuclee fizice

Determinarea numărului de nuclee fizice diferă în funcție de producătorul procesorului și de generația arhitecturii, deoarece informația poate fi raportată prin leaf-uri CPUID diferite.

Pentru procesoarele AMD, se utilizează în general leaf-urile extinse. Codul verifică mai întâi dacă procesorul suportă valoarea `CPUID` extinsă `0x80000008`. Dacă este suportată, valorile relevante sunt returnate în registrul `ECX`. Unele arhitecturi AMD folosesc biții `ECX[15:12]` pentru a indica numărul de nuclee fizice minus unu, în timp ce versiuni mai vechi utilizează `ECX[7:0]`. După interpretarea câmpului se adaugă 1 pentru a obține numărul real de nuclee.

Pentru generații mai noi, AMD oferă și leaf-ul extins `0x8000001E`, care poate furniza informații suplimentare despre topologie (spre exemplu, numărul de fire pe nucleu), permițând derivarea numărului de nuclee fizice din numărul total de fire logice.

Pentru procesoarele Intel, metodele recomandate pentru topologie sunt leaf-urile dedicate enumerării topologie: `CPUID leaf 0x0B` (Extended Topology Enumeration), `CPUID leaf 0x1F` (V2 Extended Topology Enumeration – pe procesoare mai noi)[16]. Aceste leaf-uri descriu ierarhic nivelurile de topologie (SMT și Core) și permit obținerea atât a numărului total de procesoare logice, cât și a numărului de fire per nucleu. Pe baza acestor valori , numărul de nuclee fizice poate fi determinat ca raport între totalul de procesoare logice și firele pe nucleu.

Ca metodă de rezervă, pe unele sisteme se poate utiliza și `CPUID` leaf `0x04`, unde anumite câmpuri pot indica numărul de nuclee care partajează niveluri de cache (o aproximare a nucleelor fizice), însă această metodă este mai puțin sigură decât leaf-urile de topologie.

Dacă procesorul aparține unui alt producător sau dacă leaf-urile necesare nu sunt disponibile, numărul de nuclee fizice rămâne nedeterminat.

e) Memoriile cache L1, L2, L3 (CPUID cu EAX = 0x04 și 0x80000005 / 0x80000006)

Pentru identificarea dimensiunii memoriei cache la nivelurile L1, L2, L3, codul execută o buclă care apelează `CPUID` cu `EAX = 0x04` si cu `ECX` setat succesiv cu valorile 0, 1, 2, etc. până când nu mai există alt nivel cache valid. Fiecare astfel de apel returnează informații despre un nivel cache. Biții `EAX [7:5]` indică nivelul cache-ului (1 pentru L2, 2 pentru L2, 3 pentru L3). Restul registrelor conțin dimensiunile structurale ale cache-ului: mărimea liniei de cache, numărul de partiții fizice, numărul de căi de asociativitate și numărul de seturi. Pe baza acestor valori putem calcula dimensiunea totală a cache-ului ca produsul dintre mărimea liniei de cache, numărul de partiții, numărul de căi asociative și numărul de seturi. Rezultatul este exprimat în biți și este mai apoi convertit în kilo biți.

Totodată, există arhitecturi AMD la care valoarea `0x04` pentru `CPUID` nu descrie complet toate nivelurile de cache, în special pentru L1 și L2. Pentru aceste cazuri, implementarea verifică și disponibilitatea valorilor `CPUID` extinse `0x80000005` și `0x80000006`. Dacă valoarea `CPUID 0x80000005` este suportată, registrul `ECX[31:24]` conține dimensiunea cache-ului L1 de date în kilo biți și registrul `EDX[31:24]` conține dimensiunea cache-ului L1 de instrucțiuni în kilo biți. Dacă valoarea 0x80000006 este suportată, registrul `ECX [31:16]` conține dimensiunea cache-ului L2, în kilo biți, iar registrul `EDX[31:18]` conține dimensiunea cache-ului L3 exprimat în unități de câte 512KB.

4.2 Măsurarea timpului de execuție folosind controlul de cicluri al procesorului (instrucțiunile RDTSC și CPUID)

Pentru evaluarea performanței reale a procesorului pe o anumită sarcină de lucru, codul măsoară numărul de cicluri de ceas consumate pentru executarea efectivă a calculelor. Această măsurătoare se bazează pe instrucțiunea `RDTSC` (Read Time-Stamp Counter), care citește un contor hardware incrementat la frecvența procesorului, indicând numărul de cicluri de ceas care au trecut de la pornirea calculatorului. Instrucțiunea `RDTSC` returnează un număr pe 64 de biți, împărțit între registrele EDX (partea superioară) și `EAX` (partea inferioară).

Pentru a obține o măsurătoare corectă, se folosește și instrucțiunea `CPUID` înainte de fiecare `RDTSC`. `CPUID` are efect de serializare: forțează executarea tuturor instrucțiunilor anterioare înainte să continue execuția și împiedică compilatorul din a rearanja instrucțiunile. Secvența folosită pentru măsurare este:

1. Se apelează `CPUID`.
2. Se apelează `RDTSC` și se citește valoarea inițială a contorului.
3. Se execută secțiunea de cot pe care dorim să o măsurăm
4. Se repetă instrucțiunea `CPUID`, urmată de `RDTSC` și se obține valoarea finală a contorului
5. Diferența dintre valoarea finală și cea inițială reprezintă numărul total de cicluri de ceas necesare executării secțiunii de cod.

4.3 Metoda Monte Carlo Pentru aproximarea lui π

Pentru a testa procesorul sub o sarcină de lucru numerică, codul implementează o metodă clasică tip Monte Carlo pentru estimarea valorii lui π. Ideea este următoarea: generăm puncte aleatoare (x, y) într-un pătrat de latura 1 (de exemplu x ∈ [0,1] , y ∈ [0,1]) și verificăm câte dintre ele cad în interiorul sfertului de cerc cu raza 1, adică câte satisfac ecuația $x^2 + y^2 \leq 1$. Raportul dintre punctele care satisfac ecuația și

numărul total de puncte aproximează aria sfertului de cerc. Din acest raport se poate deduce o aproximare a lui π după formula:

$$\pi \approx 4 \cdot (\frac{numă r\ puncte\ în\ cerc}{numă\ total\ puncte})$$

4.4 Generator de numere pseudo-aleatoare

Pentru generarea punctelor aleatoare utilizate în metoda Monte Carlo, codul implementează un generator propriu de numere pseudo-aleatoare, de tip Linear Congruential Generator (LCG) [13]. Acesta este un algoritm calsic de generare de secvențe de numere aparent aleatoare, folosit frecvent în benchmark-uri și simulări datorită costului său redus și comportamentului determinist.

Un generator LCG produce o nouă valoare numerică pornind de la o valoare anterioară (seed) folosind o formula de recurență de forma:

$$X_{n+1} = (a \cdot X_n + c)\ mod\ m$$

Unde:

- a este un factor de multiplicare,
- c este constanta de increment,
- m este modulul (în general o putere a lui 2, aici 2^{32}),
- X_n este valoarea curentă (seed-ul),
- X_{n+1} este valoarea nouă generată.

Deși limbajul C oferă o funcție `rand()`, aceasta nu este potrivită pentru un benchmark multi-threaded din mai multe motive, printre care faptul că `rand()` folosește o stare globală internă comună pentru toate apelurile. Dacă mai multe thread-uri ar apela `rand()` simultan, rezultatele ar fi nedeterminate sau ar putea apărea protecție cu mutex, introducând un blocaj de sincronizare pierzându-se din eficiență. În cazul aplicației, fiecare thread de execuție deține propria stare (seed) independentă.

4.5. Înmulțirea matricelor

Al doilea test de performanță implementează înmulțirea a două matrice pătratice, de dimensiune $N \times N$ în virgulă flotantă. Această operație este una dintre cele mai utilizate în domenii precum grafica 3D, simulările fizice, procesarea paralelă sau inteligența artificială, fiind totodată un test relevant pentru a evalua performanța aritmetică a procesorului.

Produsul a două matrice A și B este:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

Unde:

- A_{ik} reprezintă elementul de pe linia i, coloana k din matricea A,
- B_{kj} reprezintă elementul de pe linia k, coloana j din matricea B,

- C_{ij} este elementul rezultat în matricea produs C.

4.6. Transformata Fourier Rapidă

Al treilea benchmark implementează transformata rapidă Fourier (FFT – Fast FourierTransform). FFT este o optimizare a transformatei Fourier discrete (DFT), folosită în procesarea semnalelor audio, compresie multimedia, telecomunicații, analiză de rețele electrice și simulări numerice.

Transformata Fourier discretă a unui semnal de lungime N este definită prin:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-\frac{j2\pi kn}{N}}$$

Această formulă are o complexitate O(N^2), însă FFT reduce complexitatea la: $O(N \log_2 N)$. Prin împărțirea semnalului în componente pare și impare, se obține recurența:

$$X_k = E_k + W_k \cdot O_k$$

$$X_{\frac{k+N}{2}} = E_k - W_k \cdot O_k$$

unde $W_k = e^{-\frac{j2\pi k}{N}}$ este factorul de rotație.

4.7. Integrarea într-o bibliotecă DLL

Un DLL (Dynamic LInked LIbrary) reprezintă un modul executabil care conține funcții și date ce pot fi folosite de mai multe aplicații simultan, fără ca acestea să includă codul respectiv în executabilul propriu. Spre deosebire de bibliotecile statice (.lib), DLL-urile sunt încărcate dinamic în memorie la rulare și permit partajarea codului, reducerea dimensiunii executabilelor și modularizarea aplicațiilor.

Codul din DLL este încărcat o singură dată în memoria RAM, chiar dacă este folosit de mai multe procese. Funcțiile din DLL sunt exportate și accesibile printr-o tabelă de import (export table). Poate fi actualizat fără a recompila aplicația principală, atâta timp cât interfața rămâne aceeași. Permite separarea logici de calcul (în cadrul acestei aplicații: calculele CPU) de partea interfeței grafice(GUI, aplicație C#).

5. Design

Designul aplicației este construit în jurul unei arhitecturi modulare în care fiecare componentă are o responsabilitate clar definită și poate funcționa independent, fără a depinde de detaliile celorlalte secțiuni. Această organizare permite extinderea și reutilizarea componentelor, inclusiv integrarea bibliotecii native într-o aplicație externă. Astfel, soluția este structurată din trei părți majore: detecția de hardware, execuția benchmark-urilor și partea de interfață.

5.1. Structura generală și organizarea pe module C++

Componenta nativă este construită sub forma unei biblioteci dinamice (DLL), iar în interiorul acesteia logica este separată în module funcționale. Fiecare modul este proiectat pentru a acoperi un anumit tip de

operație: extragerea informațiilor hardware, executarea unui anumit timp de benchmark. Aceasta separare clară permite extinderea cu noi teste de performanță, dar și înlocuirea sau optimizare independentă a componentelor existente.

1. Secțiunea CPUID / Detecția hardware

Modulul de detecție hardware este responsabil pentru identificarea caracteristicilor procesorului utilizând instrucțiunea CPUID. Designul acestei componente urmărește să ofere o identificare uniformă pentru procesoare provenind de la producători diferiți, precum Intel și AMD, deși structura și modul de raportare a informațiilor pot varia semnificativ între aceste arhitecturi. În practică, modulul centralizează într-o singură structură setul complet de date relevante pentru aplicație: producătorul procesorului, denumirea comercială, numărul de nuclee fizice și fire logice, precum și dimensiunile memoriei cache pe niveluri.

Arhitectura modului prevede apeluri CPUID distincte pentru diferite tipuri de informații, folosind valori specifice ale registrului `EAX`. De exemplu, un set de apeluri este dedicat identificării producătorului, altul extragerii denumirii comerciale, iar altele pentru a determina configurația cahce-urilor și arhitectura internă a procesorului. În același timp, modulul este proiectat să detecteze automat dacă procesorul suportă un set extins de funcții `CPUID`, lucru necesar în special pentru procesoarele moderne AMD și Intel, unde unele informații sunt disponibile doar prin valori extinse. Prin acest design, detecția hardware devine robustă și adaptabilă, acoperind majoritatea arhitecturilor moderne.

2. Behcmark-ul Monte Carlo

Modulul Monte Carlo este conceput pentru a genera o sarcină intensă pe procesor, bazată pe estimarea valorii lui PI prin metoda probabilistică clasică. Designul acestui modul se concertează pe paralelizarea eficientă, astfel încât sarcina totală să poată fi împărțită în mod echilibrat între toate nucleele disponibile. Fiecare thread creat în cadrul acestui benchmark lucrează independent, folosind un generator presudo-aleator propriu, lucru care elimină necesitatea sincronizării asupra unor resurse comune și minimizează blocajele.

Sincronizarea globală a firelor de execuție reprezintă un element central al designului. Toate thread-urile sunt create în prealabil și sunt blocate până la semnalizarea unui eveniment comun de start. Această strategie elimină influențele externe și asigură faptul că măsurarea performanței reflectă strict timpul de calcul. După începerea execuției, fiecare thread procesează un număr determinat de puncte, iar rezultatele sunt apoi centralizate într-o singură estimare. Modulul este pregătit pentru scalare, astfel încât creșterea numărului de thread-uri să conducă la creșterea proporțională a performanței pe sisteme multi-core.

3. Generatorul pseudo-aleator (LCG)

Pentru a alimenta metoda Monte Carlo cu valori pseudo-aleatoare, designul include un generator liniar congruențial (LCG) minimalist, ales pentru viteza sa foarte mare și pentru a păstra independența firelor de execuție. Spre deosebire de funcțiile standard care pot avea stare gloabă și pot necesita protecție, generatorul LCG oferă un flux determinist pentru fiecare thread, pornind de la un seed diferit. Acest lucru permite o execuție complet paralelă, fără riscul de a exista coliziuni sau necesitatea unor mecanisme suplimentare de sincronizare, menținând performanța la nivel optim.

$$X_{n+1} = (1664525 \cdot X_n + 1013904223) \, mod \, 2^{32}$$

Acest generator a fost ales deoarece oferă o secvență deterministă, foarte rapidă și independentă de thread-uri. Spre deosebire de funcția standard `rand()`, acesta nu are stare globală și nu necesită mecanisme de sincronizare, eliminând riscul blocajelor.

4. Benchmark-ul de înmulțire al matricelor

Un alt modul al aplicației este dedicat benchmark-ului de înmulțire a două matrici pătratice, un test care pune accent pe accesul constant la memorie și pe calcule în virgulă flotantă. Designul acestei secțiuni presupune împărțirea uniformă a rândurilor matricei între thread-uri, astfel încât fiecare thread să fie responsabil pentru o porțiune clar delimitată din rezultatul final. În acest mod, se evită necesitatea sincronizării asupra zonelor comune de memorie și se poate obține o execuție complet paralelă.

Procesul este inițializat prin alocare și pregătirea matricelor de intrare, iar înainte de începerea calculelor toate thread-urile sunt sincronizate prin același mecanism de tip barieră utilizat și în celelalte benchmark-uri. Această uniformitate în design simplifică arhitectura generală a bibliotecii, menținând în același timp flexibilitatea necesară pentru testarea procesorului în condiții care se aproprie de aplicațiile reale.

5. Benchmark-ul transformata rapidă Fourier.

Modulul FFT este proiectat pentru a evalua performanța procesorului într-un scenariu mai complex, caracterizat de acces neuniform la memorie și calcule pe numere complexe. Designul se bazează pe algoritmul clasic Cooley-Tukey, implementat într-o manieră iterativă și organizată in-place pentru a minimiza necesarul de memorie suplimentară. Etapele algoritmului sunt structurate astfel încât anumite blocuri de date să poată fi procesate independent, ceea ce permite împărțirea sarcinii între mai multe thread-uri.

Pentru a reduce costurile în timpul execuției, factorii trigonometrico-complexi (twiddle factors) sunt pre computați și reutilizați în fiecare etapă a algoritmului. În același timp, reordonarea bit-reversal este integrată în fluxul de pregătire a datelor, contribuind la eficientizarea calculelor ulterioare. Prin această organizare, modulul FFT testează alte dimensiuni ale procesorului, diferite de cele evaluate în benchmark-urile anterioare.

5.2. DLL

Întregul cod nativ este integrat într-un DLL, ceea ce permite decuplarea dintre partea de calcul intensiv și aplicația principală. Designul bibliotecii dinamice oferă funcții exportate cu semnături simple, ușor de apelate din limbaje externe. În acest mod, aplicația poate beneficia de performanța codului nativ, fără a expune utilizatorul la complexitatea instrucțiunilor low-level sau a sincronizării între thread-uri

5.3. Interfața cu utilizatorul: C# + WinForms

Interfața grafică este proiectată astfel încât să ofere o experiență cât mai intuitivă utilizatorului. Folosirea tehnologiei WinForms este justificată prin simplitatea dezvoltării, stabilitatea mediului și integrarea facilă cu DLL-ul nativ prin mecanismul P/Invoke. Interfața se ocupă exclusiv de gestionarea

interacțiunilor cu utilizatorului, afișarea rezultatelor și coordonarea apelurilor către funcțiile exportate, menținând o separare clară față de detaliile tehnice ale benchmark-urilor.

6. Implementare

Implementarea aplicației s-a realizat în mod modular, urmărind separarea clară dintre partea de calcul (C++), care conține algoritmii și măsurătorile de performanță, și partea de interfață grafică (C# / WinForms), responsabilă cu interacțiunea cu utilizatorul și afișarea rezultatelor. Această separare este posibilă prin realizarea unei biblioteci dinamice (DLL) care conține toate funcțiile de benchmarking și detecție hardware și a unei aplicații Windows Forms care apelează funcțiile exportate din DLL prin mecanismul P/Invoke.

Scopul implementării a fost transpunerea în cod a arhitecturii propuse în etapa de design, asigurându-se: performanța maximă în partea nativă, prin folosirea de instrucțiuni low-level (CPUID, RDTSC); portabilitate și simplitatea de utilizare în partea de interfață, prin folosirea limbajului C# și a framework-ului WinForms; compatibilitatea completă între componente, ambele fiind compatibile pentru arhitecturi pe 32 de biți.

Codul nativ din DLL a fost scris în C++ și organizează funcționalitățile în patru secțiuni principale:

6.1. Detecția hardware (GetCpuInfo)

Funcția `GetCpuInfo(CpuInfo * info)` este componenta centrală pentru detecția hardware și are rolul de a popula o structură `CpuInfo` cu toate informațiile relevante despre procesor. La început, funcția verifică validitatea pointerului primit și inițializează întreaga structură cu zero, iar pentru câmpurile numerice care nu sunt încă determinate folosește valoarea -1 pentru a marca faptul că respectivul parametru nu a fost încă identificat. Această inițializare previne utilizarea unor valori reziduale în cazul unor detecții incomplete.

Primul pas în detecție îl reprezintă identificarea producătorului procesorului: Pentru aceasta, funcția execută apelul `CPUID` cu `EAX = 0`. Rezultatele din registrele `EBX`, `EDX` și `ECX` sunt copiate într-un buffer local, formând șirul numele producătorului (de exemplu „GenuineIntel” sau „AuthenticAMD”). Acest șir este apoi copiat în câmpul `vendorString` al structurii `CpuInfo`, furnizând astfel informația de bază pentru ramificările ulterioare din funcție, care se comportă diferit în funcție de producător.

Pentru a obține denumirea comercială a procesorului, funcția folosește trei apeluri succesive `CPUID` cu valorile `0x80000002`, `0x80000003` și `0x80000004`. Fiecare dintre aceste apeluri returnează câte 16 octeți de text în registrelele `EAX`, `EBX`, `ECX`, `EDX`, iar implementarea copiază aceste blocuri succesiv într-un buffer brand. După concatenarea tuturor porțiunilor, șirul este curățat de spațiile inițiale și copiat în câmpul `brandString` din structura `CpuInfo`.

Numărul de fire logice de execuție este determinat folosind apelul `CPUID` cu `EAX = 1`. Din registrul `EBX` sunt extrași biții `[23:16]`, care codifică numărul de fire de execuție. Dacă valoarea rezultată este zero, implementarea va completa cu valoarea 1, pentru a evita stocarea unei valori invalide. Câmpul `logicalThreads` este astfel setat cu numărul de thread-uri hardware pe care le are procesorul.

Determinarea numărului de nuclee fizice este implementată diferit pentru AMD și Intel. Pentru procesoarele AMD funcția verifică valoarea maximă extinsă suportată de procesor prin `CPUID(0x80000000)`. Dacă aceasta este mai mare decât `0x8000001E`, în special pentru arhitecturile mai noi, se execută `CPUID` cu `EAX = 0x8000001E`, iar din registrul `EBX` sunt extrași biții corespunzători numărului de fire pe nucleu. Pe baza acestora, se derivă numărul de nuclee fizice împărțind numărul total de fire logice la firepe pe nucleu. Pentru arhitecturi AMD mai vechi, implementarea folosește `CPUID(0x80000008)` și interpretează câmpurile din `ECX` pentru a determina numărul, aplicând la nevoie un `offset` de `+1`, conform specificațiilor. Pentru procesoarele Intel, detecția se bazează pe `CPUID(0x04)` cu `ECX = 0`, unde câmpul EAX [31:26] conține numărul de nuclee fizice minus unu. După extragere valoarea este validată și stocată în `physicalCores`.

Determinarea dimensiunilor cache-urilor se realizează printr-o combinație între valoarea standard 0x04 și valorile extinse `0x80000005` și `0x80000006`, în special pentru AMD. Funcția parcurge sub valorile pentru `EAX = 0x04` într-un ciclu, și pentru fiecare nivel de cache raportat, calculează dimensiunea în octeți pe baza formulei `lineSize*partitions*ways*sets`, convertind apoi rezultatul în KB. În funcție de nivel (`cacheLevel == 1, 2, 3`) și de tip (data, instruction sau unified), valorile sunt atribuite câmpurilor `l1DataKB, l1InstrKB, l2KB, l3KB`. Pentru procesoarele AMD, dacă după această fază unele valori nu au fost încă determinate, implementarea apelează `CPUID(0x8000005)` și `CPUID(0x80000006)`, folosind convențiile specifice (de exemplu, `ECX[31:24]` pentru L1 data, `EDX[31:24]` pentru L1 instruction, `ECX[31:16]` pentru L2 și `EDX[31:18]` pentru L3 în blocuri de 512 KB). La final, structura `CpuInfo` conține toate informațiile hardware relevante pentru aplicație.

6.2. Benchmark Monte Carlo

Benchmkar-ul MonteCarlo este implementat în jurul structurii `ThreadArgs` și al funcțiilor `MonteCarloDoWork`, `WorkerProc` și `BenchmarkMonteCarloBatch`. Structura `ThreadArgs` conține datele de intrare și de ieșire pentru fiecare thread: numărul de iterații care trebuie executate, seed-ul local al generatorului pseudo-aleator, numărul de puncte care se află în interiorul cercului (hits), precum și două handle-uri către obiecte de sincronizare de tip `HANDLE` (un eveniment comun de start și un eveniment individual de finalizare).

Funcția `MonteCarloDoWork(ThreadArgs* arg)` conține algoritmul propriu-zis de calcul pentru un thread. La început, seed-ul este preluat din structura de argumente, iar un contor local `localHits` este inițializat cu zero. În interiorul unei bucle care se execută de iterations ori, generatorul liniar congruențial actualizează seed-ul folosind relația `seed = 1664525u * seed + 1013904223u`, producând pe rând două valori întregi `rx` și `ry` care sunt apoi normalizate intervalul `[0, 1)` prin împărțirea la 2^32 și stocate în `fx` și `fy`. Pentru fiecare pereche `(fx, fy)`, funcția calculează `d2 = fx*fx + fy*fy` și incrementează `localHits` dacă punctul se află în cercul unitate (adică d2 <= 1.0f). La final, numărul de puncte din cerc este scris în câmpul `hits` al structurii, iar seed-ul final este păstrat înapoi în `arg->seed`, permițând eventuala continuare sau reproducere deterministă.

Funcția `WorkerProc(LPVOID lpParam)` reprezintă procedura de start a fiecărui thread Monte Carlo. Aceasta primește ca parametru un pointer la structura `ThreadArgs`, pe care asignează la tipul corespunzător. Primul pas al funcției este să aștepte semnalizarea evenimentului global de start (`startEvent`) prin `WaitForSingleObject`, ceea ce garantează ca toate firele rămân blocate până când thread-ul principal decide să pornească efectiv benchmark-ul. După deblocare, `WorkerProc` apelează `MonteCarloDoWork` pentru a efectua calculele, apoi semnalizează propriul eveniment de finalizare (`doneEvent`) prin `SetEvent`, atenționând astfel thread-ul principal că a terminat.

Funcția `BenchmarkMonteCarloBatch(unsigned int numThreds)` orchestrează întregul proces al benchmark-ului MonteCarlo, de la inițializarea thread-urilor și a obiectelor de sincronizare până la măsurarea timpului total de execuție în cicluri de procesor. În primul rând, funcția face în așa fel încât `numThreads` să fie cel puțin 1 și cel mult `MAX_THREADS`. Apoi creează un eveniment de tip manual-reset (`startEvent`) pe care îl vor aștepta toate thread-urile și, pentru fiecare thread creează un eveniment individual de finalizare, un `ThreadArgs` și un `HANDLE` de thread prin `CreateThread`. Sarcina totală definită de constantă `TOTAL_ITERATIONS`, este împărțită la în mod uniform între thread-uri prin calcului unui `baseIter` și al unui rest (`leftover`), astfel încât ultimul thread să preia eventualele iterații suplimentare. Seed-ul fiecărui thread este generat pe baza lui `clock()` și a unui factor de amestecare, pentru a evita corelații evidente între threawd-uri.

Măsurare timpului se realizează folosind un șablon comun tutor benchmark-urilor. Înainte de secțiunea temporizată, funcția execută de mai multe ori combinația `cpuid + rdtsc` pentru a pregăti pipeline-ul și pentru a măsura separat costul instrucțiunii `CPUID` stocat în variabila `cpuid_time`. Apoi sunt citite timbrele de timp înainte și după zona de lucru (delimitată de apelul `SetEvent(startEvent))` și de `WaitForMultipleObjects` care așteaptă terminarea tuturor thread-urilor. Cele două valori rezultate (superior și inferior în registrele `EDX` și `EAX`) sunt combinate într-un `unsigned __int64`, iar diferența dintre ele, din care se scade `cpuid_time`, reprezintă numărul total de cicluri consumate de benchmark. La final, funcția eliberează toate handle-urile de thread și de eveniment și returnează `total_cycles`.

6.3. Benchmark-ul de înmulțire a matricilor

Benchmark-ul de înmulțire a matricelor este construit în jurul structurii `ThreadArgsMatrix` și funcțiilor `MatrixMulDoWork`, `MatrixWorkerProc` și `BenchmarkMatrixBatch`. Structura `ThreadArgsMatrix` conține pointeri către cele trei matrice implicate în operație (A, B și C), dimensiunile matricelor pătratice num, precum și informații despre intervalul de rânduri pe care trebuie să îl proceseze fiecare thread: startRow (primul rând incluziv) și rowCount (numărul de rânduri. Similar cu benchmark-ul Monte Carlo, structura mai include un startEvent comun și un doneEvent individual pentru sincronizare.

Funcția `MatrixMulDoWork(ThreadArgsMatrix* arg)` realizează operația de înmulțire pentru subsetul de rânduri alocat unui thread. Pe baza parametrilor `startRow` și `rowCount`, funcția calculează intervalul [iStart, iEnd) și verifică că acesta este unul corect în raport cu dimensiunea matricei. Pentru fiecare rând i din acest interval, se obține un pointer către rândul corespunzător din matricea C, repsectiv un pinter către rândul din A. Apoi se parcurge fiecare coloană j din matricea rezultat și se

calculează elementul `C[i, j]` ca sumă a produselor `Ai[k] * B[k * num + j]` pentru k de la 0 la num-1. Aceasta corespunde implementării clasice a algoritmului de înmulțire a două matrice pătratice dense, fără optimizări de tip blocare sau vectorizare explicită, ceea ce este însă suficient pentru a genera o sarcină numerică intensă.

Procedura de thread `MatrixWorkerProc(LPVOID lpParam)` urmează același model ca în cazul Monte Carlo. Fiecare thread așteaptă mai întâi ca evenimentul global de start să fie semnalizat, prin WaitForSingeOBject, apoi apelează MatrixMulDoWork cu argumentele corespunzătoare și la final, semnalizează evenimentul său doneEvent pentru a indica faptul că și-a terminat partea de calcul. Astfel, se asigură ca toate thread-urile încep efectiv multiplicarea de matrice în același moment, permițând o măsurare corectă a timpului.

Funcția `BenchmarkMatrixBatch(Unsigned int numThreads)` este responsabilă de alocarea și inițializarea matricelor, de crearea thread-urilor și de măsurarea timpului de execuție. Dimensiunea matricelor este determinată de constanta `MM_NR`, iar memoriile pentru `A`, `B` și `C` sunt alocate prin `_algined_malloc` cu aliniere la 64 de byes, fapt care favorizează accesul optim la memorie și utilizarea liniilor de cache. Matricele `A` și `B` sunt apoi populate cu valori deterministe, în funcție de indecșii de linie și de coloană, în timp ce C este inițializată cu zero. În cazul în care alocarea oricărei dintre matrice eșuează, funcția eliberează resursele deja alocate și transmite un un cod de eroare negativ.

Împărțirea sarcinii între thread-uri se face prin calculului unui număr de rânduri de bază (`baseRows`) pe thread și a unui rest (`leftover`). Primele thread-uri primesc câte un rând suplimentar din rest, pentru a asigura o distribuție cât mai uniformă. Pentru fiecare thread, funcția construiește un `ThreadArgsMatrix`, setează pointeri către matrice, dimensiunea num, rândul de început și numărul de rânduri, precum și evenimentele de start și de finalizare, apoi creează thread-ul corespunzător prin `CreateThread`. Mecanismul de temporizare folosește exact același șablon cu `CPUID` și `RDTSC` ca în benchmark-ul MonteCarlo. La final toate handle-urile de thread și eveniment sunt închise, matricele sunt eliberate cu _aligned_free, iar `BenchmarkMatrixBatch` returnează numărul de cicluri necesare executării benchmark-ului.

6.4. Benchmark-ul FFT (Fast Fourier Transform)

Benchmark-ul FFT este cel mai complex dintre testele implementate și este construit pe baza mai multor funcții auxilare și a structurii `ThreadArgsFFT`. Aceeastă structură conține pointer către buffer-ele de intrare și ieșire pentru partea reală și imaginară (real și imag), dimensiunile transformatei nr, indicii de bloc de la care un thread trebuie să înceapă procesarea (`startBlock`) și numărul de blocuri pe care le procesează (`blockCount`). În plus, structura păstrează pointeri către tablourile de factori de rotație pre computați (`twRe` și `twIm`), precum și handle-urile pentru evenimentele de sincronizare `startEvent` și `doneEvent`.

Funcția `ReverseBits` este un utilitare care inversează ordinea biților unui indice, fiind utilizată în reordonarea bit-reversal necesară pentru FFT radix-2. Pe baza acesteia, funcția `BitReverseReorder` parcurge toate pozițiile de date de al 0 la nr-1, calculează poziția cu biți inverși j pentru fiecare indice i, și în cazul în care j > i, interschimbă elementele corespunzătoare din vectorii real și imaginar. Astfel, datele de intrare sunt reordonate într-o formă care facilitează execuția ulterioară a etapelor FFT.

Precalcularea factorilor de rotație este realizată de funcția `PrecomputeTwiddles`, care pentru fiecare k din intervalul [0, nr/2), calculează valoarea exp(-2πi k / nr) și memorează separat partea reală și cea imaginară în tablourile `twRe` și `twIm`. Prin realizarea aceste precomputări, FFT propriu-zis nu mai este nevoie să evalueze funcțiile trigonometrice la fiecare pas, reducând semnificativ overhead-ul de calcul.

Funcția `FFT_Block` implementează efectiv algoritmul FFT radix-2, într-o variantă iterativă și in-place. Mai întâi, datele sunt reordonate prin `BitReverseReorder`. Apoi, pentru fiecare mărime de „fluture" m (care pornește de la 2 și se dublează până la nr), se calculează half = m / 2 și un pas step în tabloul twiddles, proporțional cu raportul dintre dimensiunea totală nr și m. Pentru fiecare offset k din [0, half), se selectează factorul de rotație corespunzător și se aplică operațiile de fluture pe toate perechile de elemente (i0, i1) spațiate la distanța half în cadrul blocului de lungime m. În acest fel, FFT-ul este construit în etape, fiecare etapă combinând rezultate intermediare din etapa anterioară, conform structurii clasice Cooley-Tukey.

Funcția `FFTDoWork(ThreadArgsFFT* arg)` aplică `FFT_Block` la un număr de blocuri consecutive din buffer-urile `real` și `imag`, în funcție de parametrii `startBlock` și `blockCount`. Fiecare bloc este constituit din nr elemente și este tratat independent, ceea ce permite împărțirea naturală a volumului de lucru între thread-uri. Procedura de thread `FFTWorkerPRoc` urmează aceeași schemă de sincronizare ca și celelalte benchmark-uri.

Funcția `BenchmarkFFTBatch(unsigned int numThreads)` coordonează întregul benchmark FFT. După verificare numărului de thread-uri, sunt calculate dimensiunea transformatei nr (dată de FFT_N) și numărul total de blocuri `BLOCKS` constanta `FFT_BLOCKS`). Se alocă, prin `_algined_malloc`, buffer-ele pentru partea reală și imaginară a datelor, precum și tablourile pentru factorii de rotație, toate aliniate la 64 de biți pentru o mai bună performanță. Întregul set de date este apoi inițializat cu valori pseudo-aleatoare generate printr-o variantă simplă de LCG, în timp ce parte imaginară este setată la zero. Twiddle-uriel sunt pre computate prin `PrecomputeTwiddles`.

Distribuirea blocurilor între thread-uri se face prin împărțirea lui `BLOCKS` la `numThreads`, definind un număr de blocuri de bază pe thread (`blocksPer`) și un rest (`extra`). Fiecărui thread i se atribuie un număr de blocuri egal cu `blocksPer` plus eventual un boc suplimentar dacă indexul său este mai mic decât extra. Variabila cursor marchează offset-ul de bloc de la care începe fiecare thread. Pentru fiecare thread, sunt completate câmpurile din `ThreadArgsFFT`, sunt create evenimentele proprii de finalizare și este pornit thread-ul prin `CreateThread`. În cazul în care crearea unui thread eșuează implementarea semnalizează direct `doneEvent` pentru acel thread.

Măsurarea timpului în `BenchmarkFFTBatch` utilizează același mecanism bazat pe `CPUID` și `RDTSC` ca și celelalte benchmark-uri. La final, toate handle-urile sunt închise, iar buffer-ele alocate sunt eliberate. Funcția returnează numărul total de cicluri, care reprezintă durata de execuție a întregului `benchmarkFFT` asupra întregului set de date.

6.5. Impelentarea DLL-ului

Partea nativă a aplicației este livrată sub forma unei biblioteci dinamice (DLL), construită ca un proiect separat de tip „Dynamic Link Library” în cadrul mediului Visual Studio. Implementarea efectivă a DLL-ului este una directă: fișierul sursă C++ care conține codul pentru detecția hardware și pentru cele trei benchmark-uri a fost preulat din proiectul inițial de tip aplicație C++ și inclus în proiectul DLL. Astfel, algoritmii și logica de măsurare existente au fost reutilizate fără modificări majore, diferența principală constă în modul în care sunt compilate și expuse.

Proiectul DLL este configurat pentru arhitectura pe 32 de biți, astfel încât să fie compatibil cu aplicația C# care îl încarcă ulterior. În cazul acestui proiect, funcțiile de interes sunt declarate în fișierul de `CpuBnech.h` cu specificatorul de export, astfel încât să devină vizibile din afara bibliotecii. Compilarea proiectului generează fișierul `CpuBenchDll.dll`, care conține toate aceste funcționalități împachetate într-un modul unic, independent de interfața grafică. După compilarea proiectului DLL; fișierul rezultat este copiat în directorul aplicației C# (`CPUBench.app`) alături de executabilul WinForms. În acest fel, la rulare aplicația poate încărca și apela funcțiile native direct din DLL, fără pași suplimentare de instalare sau configurare.

6.6. Implementarea interoperabilității cu DLL în C#

Componenta C# este responsabilă pentru comunicarea cu biblioteca și pentru reprezentarea rezultatelor către utilizator. Interacțiunea cu DLL-ul `CPUBenchDll.dll` este realizată printr-o clasă static internă denumită native, în care sunt declarate prototipurile funcțiilor native cu ajutorul atributului DllImport. Fiecare funcție exportată din DLL (`GetCpuInfo`, `BenchmarkMonteCarloBatch`, `BenchmarkMatrixBatch` și `BenchmarkFFTBatch`) este mapată la o metodă statică C# cu aceeași semnătură logică, folosind convenția de apel `CallingConvention.Cdecl` pentru a corespunde implementării C++.

Pentru a permite transferul de informații despre procesor între partea nativă și codul C#, structura CpuInfo este redefinită în C# folosind atribute de interoperabilitate. Structura este decorată cu `[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Ansi)]`, ceea ce garantează compatibilitatea layout-ului în memorie cu structura C++ corespondentă. Câmpurile `brandString` și `vendorString` sunt declarate ca șiruri de caractere cu lungime fixă, prin atributele `[MarshalAs(UnmanagedType.ByValTStr, SizeConst = 64/16)]`, astfel încât vectorii de caractere din C++ să fie corect interpretați în .NET. Câmpurile numerice (numărul de nuclee fizice, fire logice, dimensiunile cache-urilor) sunt reprezentate prin tipul `int`, corespunzând tipurilor întregi din structura nativă. Astfel, apelul `Native.GetCpuInfo(ref info)` completează câmpurile unui obiect `CpuInfo` gestionat, fără a fi necesare conversii manuale sau copieri suplimentare.

Maparea funcțiilor de benchmark la metode C# urmează aceeași idee: fiecare metodă nativă este declarată ca funcție statică ce primește un parametru de tip uint pentru numărul de thread-uri și returnează un `long`, corespunzând timpului `__int64` din C++. Codul C# folosește aceste metode ca pe orice late funcții obișnuite, fără a fi conștient de implementarea din spate, iar logica de orchestrare și afișare a rezultatelor este rezolvată exclusiv în partea gestionată.

6.7. Implementarea interfeței grafice în C#

Interfața grafică a aplicației este implementată în C# folosind Windows Forms și este centrată în jurul unei singure ferestre principale. Aceasta combină trei zone funcționale: afișarea informațiilor despre procesor, configurarea și lansarea benchmark-urilor și vizualizrea rezultatelor, atât sub formă de text, cât și grafică. Layout-ul general este realizat cu ajutorul unui tabel de tip `TableLayoutPanel`, care împarte fereastra în secțiuni orizontale: prima pentru specificațiile hardware, a doua pentru controalele de configurare a testelor și a treia pentru zona de rezultate.

La încărcarea ferestrei, logica de inițializare interoghează mai întâi biblioteca nativă pentru a obține informațiile despre procesor. Se creează o instanță a structurii gestionate `CpuInfo`, aceasta este trecută prin referință către funcția nativă `GetCpuInfo`, iar rezultatele sunt preluate și afișate în elementele de tip etichetă din partea superioară a ferestrei. Utilizatorului îi sunt prezentate astfel producătorul, modelul procesorului, numărul de nuclee și fire logice, precum și dimensiunii cache-urilor, oferind un context clar pentru interpretarea benchmark-urilor. În același timp, combobox-urile destinate selectării algoritmului și a numărul de thread-uri sunt populate cu valori implicite, astfel încât utilizatorul să poată porni rapid un test fără configurări suplimentare.

Logica de interacțiune cu benchmark-urile este organizată în jurul a două butoane principale. Unul declanșează o „rulare completă”, în care toți algoritmii disponibili sunt executați pentru mai multe configurații de thread-uri, iar celălalt permite rularea personalizată a unui singur algoritm, cu un număr de thread-uri ales de utilizator. Pentru a separa clar partea de UI de logica de testare, aplicația definește un tip enumerat intern care reprezintă algoritmii disponibili și metode de utilizare care transformă selecțiile din combobox-uri în valori de acest tip și numere întregi. Executarea propriu-zisă a unui benchmark este încapsulată într-o metodă dedicată, care în funcție de algoritmul ales, apelează funcția corespunzătoare din DLL și returnează numărul de cicluri obținut.

Rularea benchmark-urilor este realizată asincron, folosind task-uri pe fir separat, pentru a evita blocare interfeței grafice în timpul execuției testelor, care pot dura mai mult. Butoanele și controalele de configurare sunt temporar dezactivate pe durata unui test, iar la final sunt reactivate, astfel încât utilizatorul să nu poată lansa accidental mai multe rulări suprapuse. Rezultatele sunt afișate într-o zonă de text, sub forma unor mesaje care indică algoritmul, numărul de thread-uri, numărul de cicluri măsurate și nota obținută. Pentru o experiență mai plăcută, aplicația include și o animație simplă care actualizează periodic textul butonului de rulare pentru a sugera că benchmark-ul este în curs de desfășurare. Pe lângă afișarea textului, aplicația oferă și o reprezentare grafică a rezultatelor, folosind un control de tip grafic integrat în zona inferioară a ferestrei. Pentru fiecare algoritm, se construiește o serie de date în care axa orizontală reprezintă numărul de thread-uri, iar axa verticală numărul de cicluri CPU, valorile fiind conectate prin linii și marcate cu puncte. Acest grafic permite compararea vizuală a scalării cu numărul de fire de execuție pentru fiecare benchmark în parte și evidențiază ușor diferențele de comportament între algoritmi. Organizarea în două panouri – unul pentru text, celălalt pentru grafic – oferă o perspectivă dublă asupra aceleași informații, făcând aplicația mai ușor de folosit și de interpretat.

7. Aplicația

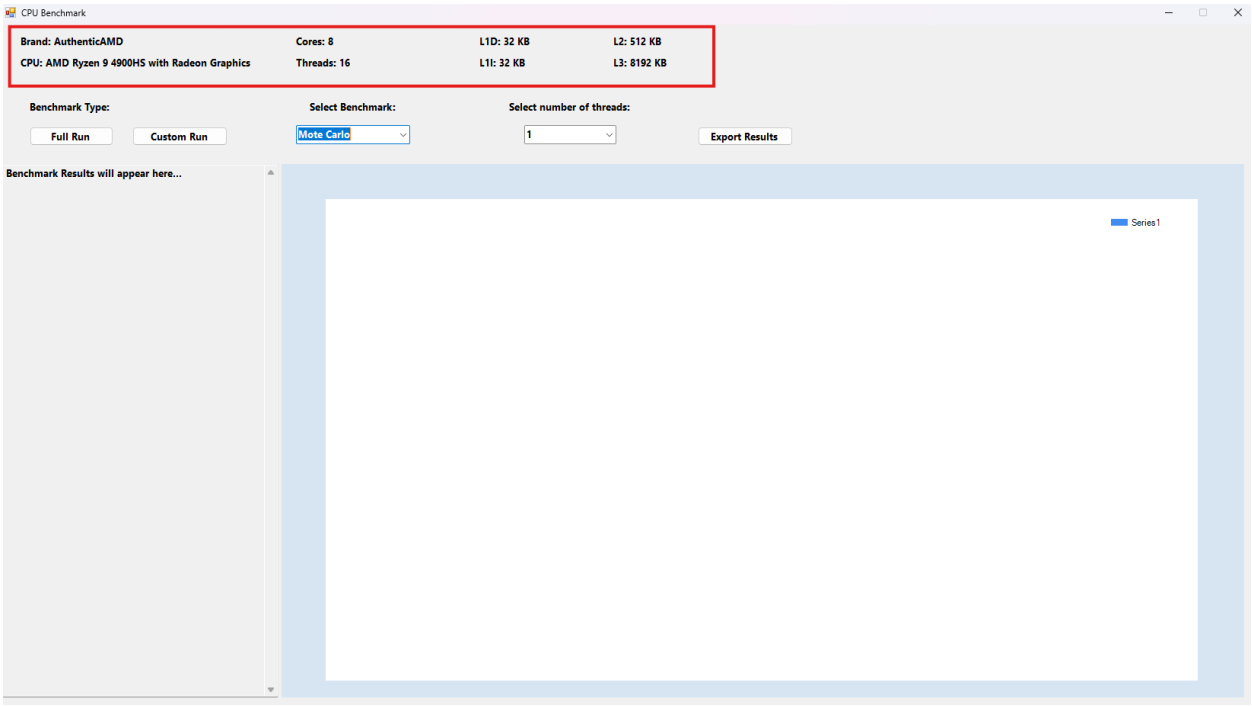
Aplicația grafică realizată în C# oferă o interfață prietenoasă interfață grafică pentru a putea rula benchmark-urile implementate în bibliotecă. Interfața are rolul de a centraliza toate funcțiile principale ale proiectului: afișarea informațiilor referitoare la hardware-ul procesorului, configurarea testelor, lansarea benchmark-urilor și vizualizarea rezultatelor în format text și grafic. Structura aplicației este împărțită în trei zone principale: zona de specificații hardware (partea superioară), zona de configurare și control al benchmark-urilor (centru-sus) și zona de rezultate (stâng pentru text și dreapta pentru grafice.

Scopul interfeței este de a permite utilizatorului să folosească teste complexe, implementate în C++, fără a fi necesare cunoștințe tehnice avansate.

7.1. Zona de afișare a informațiilor despre procesor

În partea superioară a aplicației este afișată o secțiune informativă care extrage automat, la pornirea programului, date despre procesor folosind funcția GetCpuInfo din DLL. Informațiile sunt prezentate sub formă de etichete:

1. Brand – producătorul procesorului (ex. „AuthenticAMD” sau „GenuineIntel”)
2. CPU – denumirea comercială completă
3. Cores – numărul de nuclee fizice detectate
4. Threads – numărul de fire logice disponibile
5. L1D, L1I, L2, L3 – dimensiunile cahce-urilor procesorului



Figură 1 Zona de afișare a informațiilor despre procesor

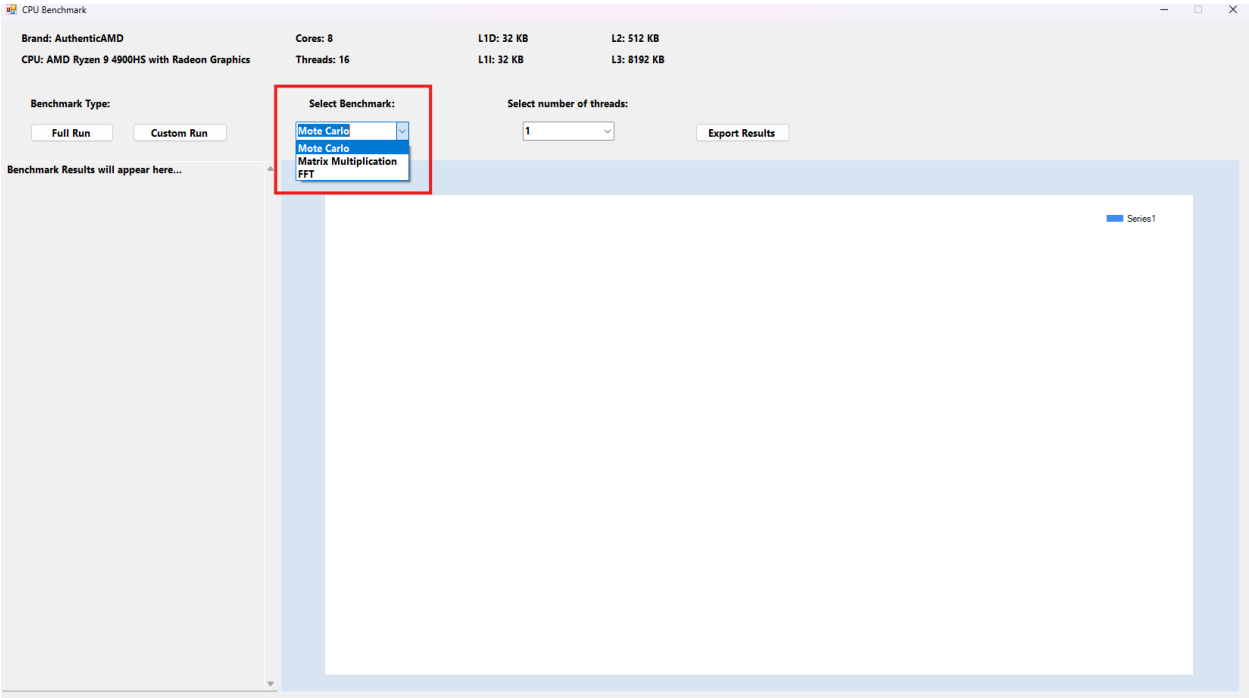
Această secțiune oferă contextul necesar pentru a putea interpreta rezultatele din benchmark-uri, deoarece principalul motiv pentru care performanța diferă este arhitectura și numărul de nuclee ale sistemului.

7.2. Configurarea Benchmark-urilor

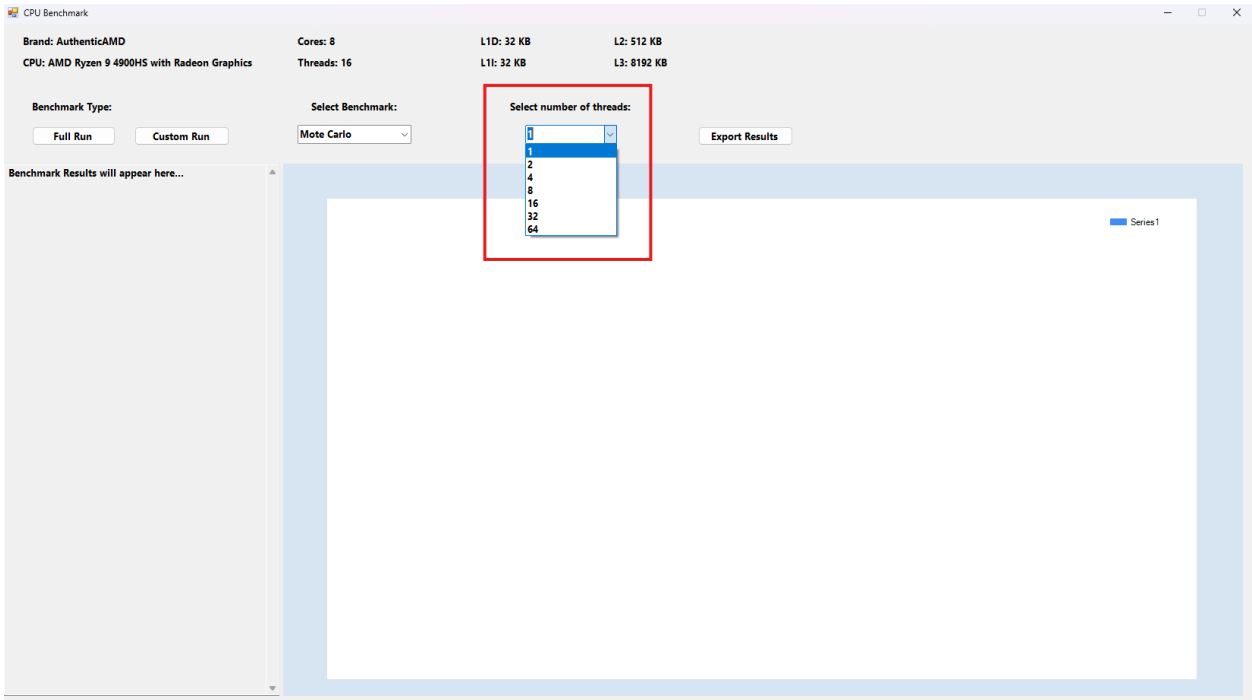
Zona centrală a aplicației conține controalele care permit configurarea testelor:

- Select Benchmark: listă din care utilizatorul poate alege tipul de test: Monte Carlo, Matrix Multiplication sau FFT.
- Select number of threads: permite alegerea numărului de thread-uri cu care va rula algoritmul (1, 2, 4, 8, 16, 32 sau 64).

Prin intermediul acestor controale utilizatorul poate configura modul în care benchmark-urile sunt distribuite pe mai multe nuclee ale procesorului, lucru important pentru a analiza scalarea paralelizării.



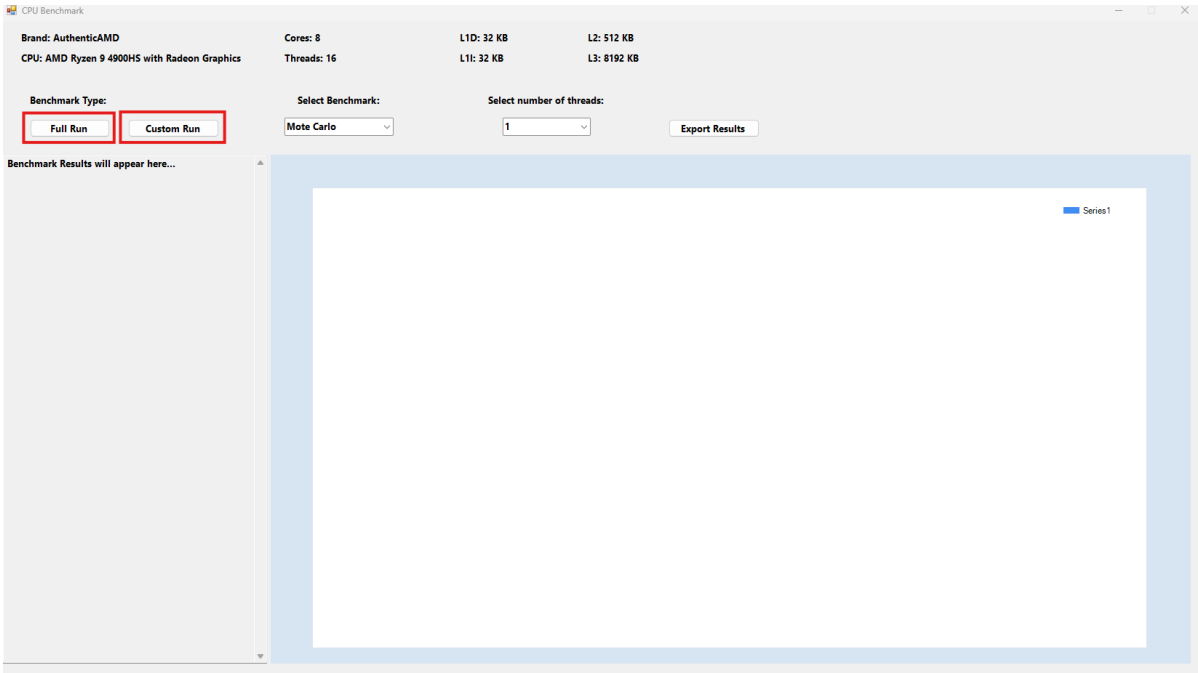
Figură 2 Selectarea tipurilor de benchmark



Figură 3 Selectarea numărului de fire de execuție

Două butoane principale care controlează execuția:

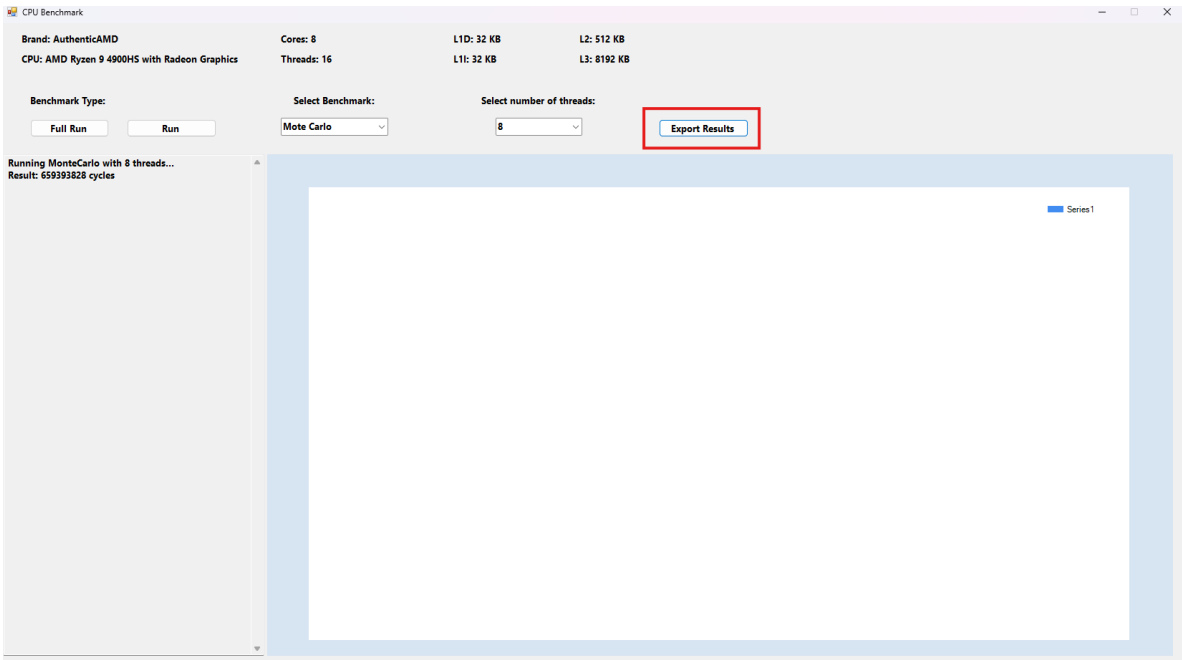
- Full Run – execută toate cele trei benchmark-uri pentru un set complet de valori ale numărului de thread-uri (de la 1 la 64). Rezultatele sunt acumulate și afișate la final în ordine.
- Custom Run – Rulează doar benchmark-ul selectat, cu numărul de thread-uri selectat.



Figură 4Butoanele Full Run și Custom Run

În timpul executării testelor, butoanele sunt dezactivate pentru a preveni rulări simultane, iar textul butonului Full Run schimb animat afișajul („Running...”) pentru a indica rularea benchmark-urilor.

De asemenea, există un buton prin care utilizatorul poate exporta rezultatele obținute în urma rulării benchmark-urilor. Acesta se poate apăsa abia după ce a fost rulat cel puțin un benchmark.

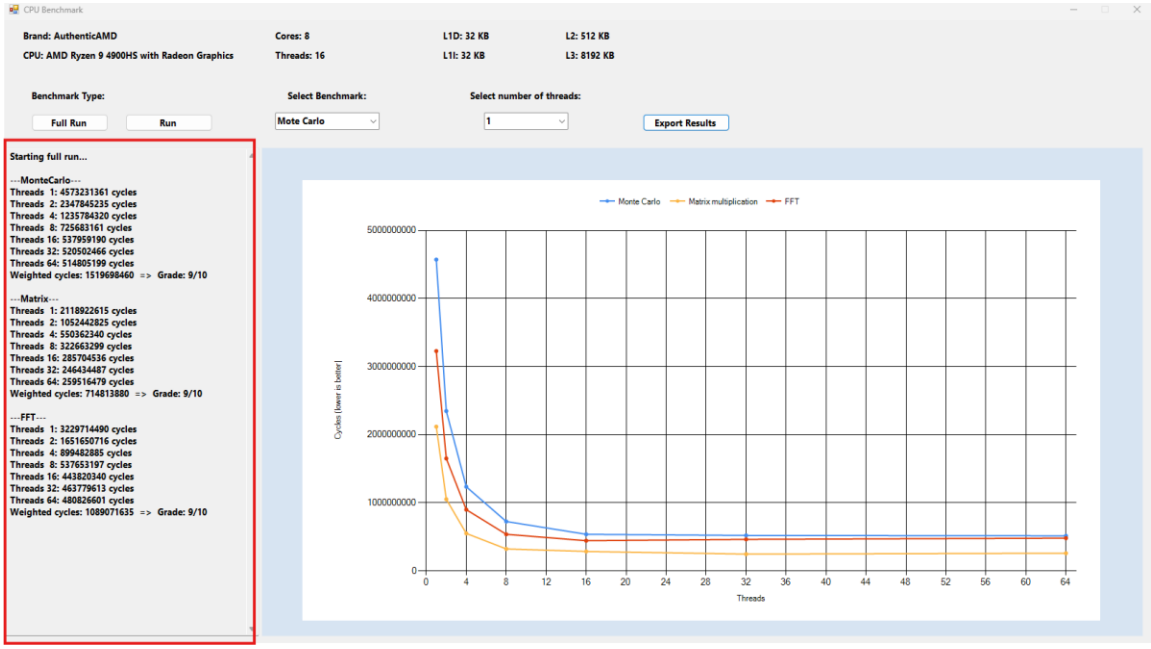


Figură 5 Buton pentru exportarea rezultatelor

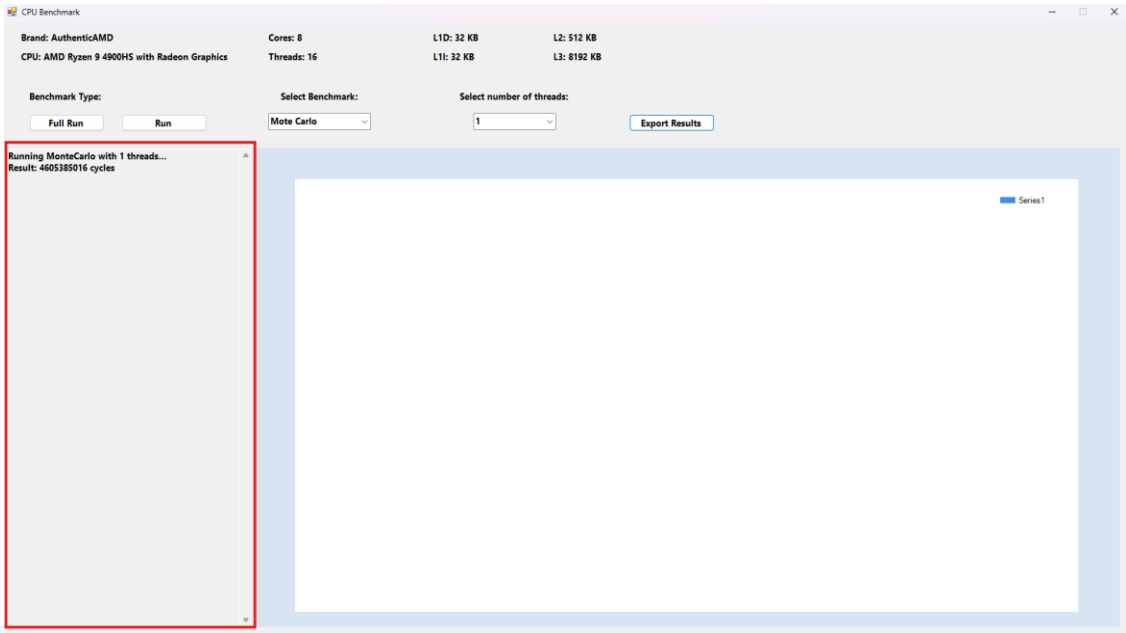
7.3. Zona de rezultate

Textul rezultatelor (partea stângă)

Această zonă afișează valorile returnate de benchmark-uri, adică numărul de cicluri CPU necesare pentru execuția fiecărui benchmark. Datele sunt structurate după benchmark și după numărul de thread-uri. Formatul textului permite citirea ușoară a comportamentului procesorului și compararea dintre teste.



Figură 6 Zona de rezultate pentru Full Run



Figură 7 Zona de rezultate pentru Custom Run

Pentru a putea exporta rezultatele obținute în partea din stânga în urma rulării benchmark-urilor în oricare dintre moduri a fost adăugat un buton în zona de configurare „Export Results”, care deschide un dialog prin care utilizatorul poate să salveze textul în format .txt sau .csv. Acest buton nu este disponibil până în momentul rulării a cel puțin unui benchmark, după această el va scrie în fișier doar rezultatele ultimului benchmark rulat, nu o istorie a acestora.

Graficul (partea dreaptă)

Graficul afișează evoluția numărului de cicluri în funcție de numărul de thread-uri. Axa X reprezintă numărul de thread-uri, iar axa Y numărul de cicluri CPU (unde valori mai mici indică performanțe mai bune).

Fiecare benchmark este reprezentat printr-o curbă cu marcaje distincte. Acest mod de vizualizare permite observarea imediată a scalării, scăderea curbei indicând o distribuție eficientă a sarcinii pe mai multe nuclee, iar aplatizarea evidențiază limitele de paralelizare ale algoritmului și ale procesorului.

Figură 8 Zona de grafic

7.4. Modul de utilizare al aplicației

Utilizatorul primește două fișiere pe un stick USB:

- CpuBench.app.exe (aplicația grafică)
- CpuBenchDLL.dll (biblioteca)

Ambele trebuie plasate în același director, altfel aplicația nu poate încărca funcțiile. Pașii pentru rularea aplicației și a unui test sunt următorii:

1. Lansezi CpuBench.app.exe.
2. Aplicația afișează automat informațiile despre procesor.
3. Selectezi tipul de benchmark
4. Alegi numărul de thread-uri.
5. Aleși Custom Run pentru un singur test sau Ful Run pentru Rularea completă.
6. Rezultatele apar în stânga, iar graficul se actualizează automat în dreapta dacă este cazul.

În unele situații, Windows Defender poate semnala executabilul ca necunoscut, deoarece aplicația nu are certificat digital sau licența comercială. Acesta este un comportament normal pentru aplicațiile compilate manual. În acest caz, utilizatorul poate selecta opțiunea „Run anyway” pentru a putea continua rularea programului.

8. Testare

Testarea aplicației a avut drept scop verificarea funcționării corecte a programului și evaluarea performanțelor procesorului în diferite scenarii. Pentru a obține rezultate comparabile între sisteme,

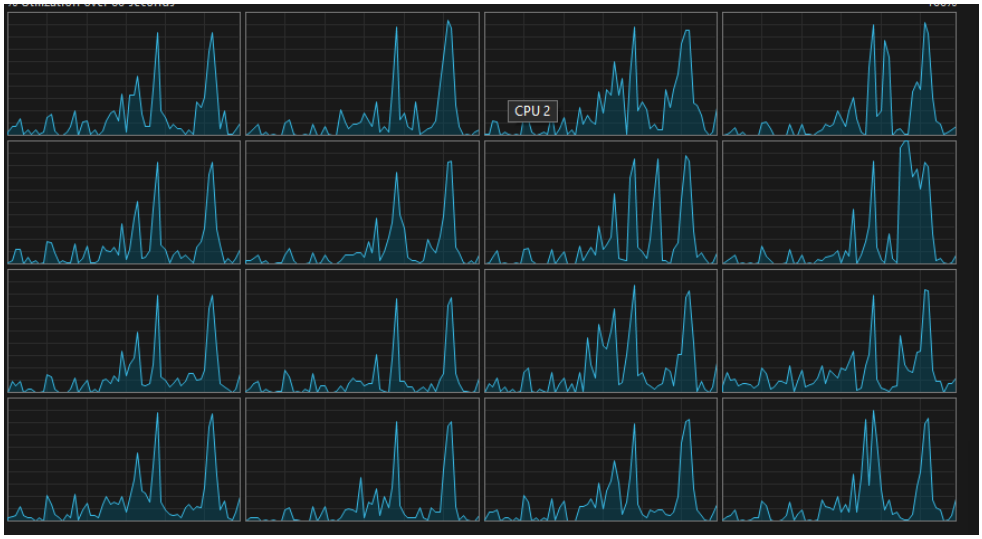
măsurarea performanței s-a realizat exclusiv în cicluri de ceas, iar apoi aceste valori au fost convertite în secunde, folosind o frecvență estimată a procesorului pentru a facilitat interpretarea rezultatelor.

Procesul de testare a fost împărțit în două etape: o testare extinsă pe sistemul propriu, unde fiecare benchmark a fost rulat de 100 de ori pentru a obține valori mediate cu o variabilitate redusă, și o testare suplimentară pe mai multe calculatoare, destinată comparației între arhitecturi hardware diferite. Rezultatele acestor teste vor fi prezentate și analizate în secțiunile următoare, iar ulterior vor fi folosite ca bază pentru realizarea unui sistem de notare a procesoarelor.

8.1. Testarea pe sistemul propriu

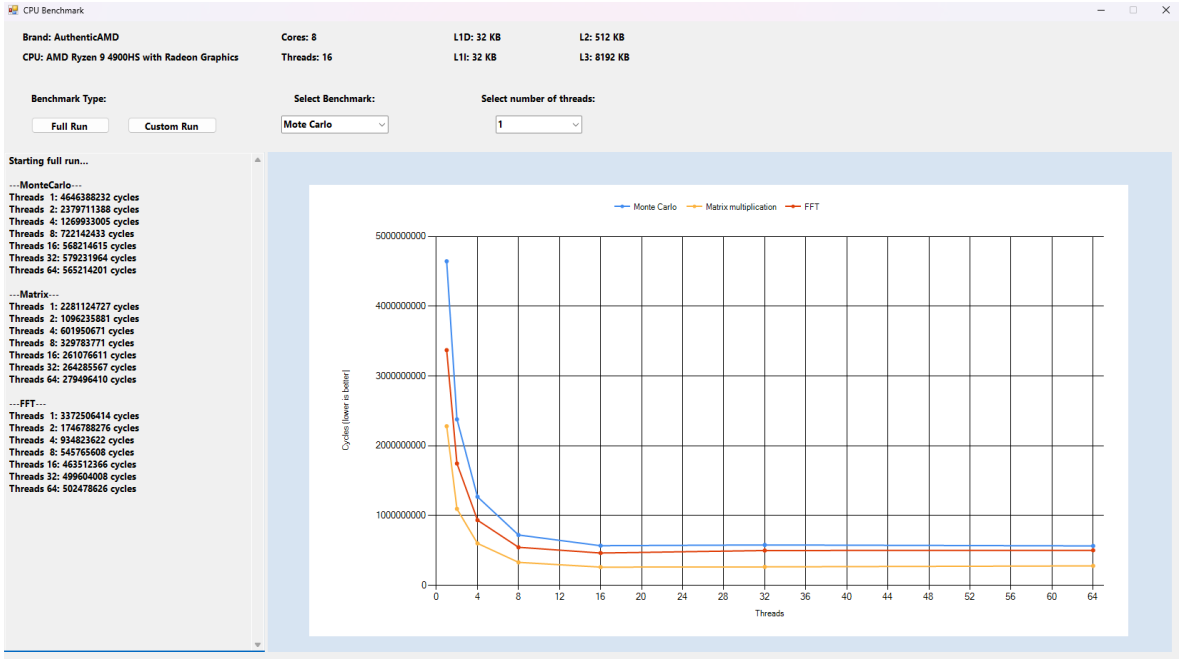
Testarea inițială a fost realizată pe sistemul propriu, reprezentat de un laptop ASUS ROG Zephyrus G14 (model GA401IV), echipat cu un procesor AMD Ryzen 9 4900HS, cu 8 nuclee și 16 fire de execuție, având o frecvență maximă de 4.40 GHz, 16 GB memorie DDR4, un SSD NVMe de 1TB, precum și o placă grafică dedicată NVIDIA GeForce RTX 2060 cu 6GB VRAM.

Rularea benchmark-urilor pe sistemul propriu a fost monitorizată cu Task Manager pentru a observa distribuția sarcinii pe nuclee. Utilizarea procesorului prezintă spike-uri clare la fiecare secțiune de test, confirmând că thread-urile sunt lansate simultan și că aplicația utilizează corect resursele hardware disponibile.



Figură 9 Task manager din timpul execuției a unui singur rând de benchmark-uri.

Pentru a obține valori relevante, benchmark-urile au fost rulate de 100 de ori pe calculatorul propriu. Această etapă are rolul de a elimina posibilele variații generate de sistemul de operare și de procesele care rulează în fundal. După adunarea datelor bute, valorile au fost mediate și stau la baza graficelor finale, care reflectă mult mai fidel comportamentul algoritmilor decât o singură rulare care este susceptibilă la interferențe.



Figură 10 Rezultatele rulării benchmark-urilor de 100 de ori înainte de adăugarea notelor și a butonului de export

	Monte Carlo		Matrix Multiplication		Fast Fourier Transform	
	Cicluri	Secunde	Cicluri	Secunde	Cicluri	Secunde
1	4646388232	1.2557	2281124727	0.6165	3372506414	0.9114
2	2379711388	0.6431	1096235881	0.2962	1746788276	0.4721
4	1269933005	0.3432	601950671	0.1626	934823622	0.2526
8	722142433	0.1951	329783771	0.0891	545765608	0.1475
16	568214615	0.1535	261076611	0.0705	463512366	0.1252
32	579231964	0.1565	264285567	0.0714	499604008	0.1350
64	565214201	0.1527	279496410	0.0755	502478626	0.1358

Pentru a transforma numărul de cicluri în secunde, a fost necesară estimarea frecvenței reale la care a funcționat procesorul în timpul execuțiilor. Observațiile din Task manager indică faptul că procesorul a rulat predominant la o valoare intermediară între frecvența de bază (3.0 GHz) și frecvența maximă (4.40 GHz). Astfel, pentru conversie a fost utilizată o frecvență medie estimată de 3.7 GHz. Această valoare nu reflectă o măsurătoare hardware exactă, ci o aproximare realistă necesară pentru comparabilitate între teste.

8.2. Testarea pe sisteme suplimentare

Pentru sistemele care au fost testate, frecvența a fost fie estimată pe baza mediei între frecvența de bază și cea maximă oferite de producător în specificațiile sistemului, fie folosind frecvența maximă ajustată cu o marjă de 0.3-0.4 GHz, pentru a compensa fluctuațiile dinamice ale frecvenței. Scopul acestor conversii este de a obține o estimare comparabilă a performanței, întrucât valorile exacte pot varia semnificativ în funcție de mecanismele fiecărui procesor.

După finalizarea testării pe sistemul propriu, benchmark-urile au fost rulate și pe alte calculatoare. Pe aceste sisteme, din motive de timp și disponibilitate a resurselor, fiecare benchmark a fost rulate de 10 ori, iar valorile finale reprezentate în tabele reprezintă media acestor execuții. Deși numărul de repetări este mai redus decât în cazul sistemului propriu, rezultatele obținute sunt suficient de stabile și permit o comparație relevantă între configurațiile hardware analizate.

Calculatoare pe care s-au realizat testele pentru aplicație sunt:

Sistem	Procesor	Nuclee	Fire de execuție	Frecvență	RAM
Laptop 1	AMD Ryzen 9 4900HS	8	16	3.0-4.4GHz	16 GB
Laptop 2	AMD Ryzen 7 6800HS	8	16	3.2 – 4.7 GHz	16 GB
Laptop 3	Intel(R) Core(TM) i5-1135G7	4	8	2.4 – 4.2 GHz	24 GB
Laptop 4	AMD Ryzen 7 7745HX	8	16	3.6 – 5.1 GHz	16GB
Laptop 5	AMD Ryzen 7 3750H	4	8	2.3 – 4.0 GHz	8 GB
Laptop 6	Intel(r) Core(TM) i3-1005G1	2	4	1.20 - 3.4 GHz	4 GB

Rezultatele pentru Algoritmul Monte-Carlo:

Threads	Laptop1	Laptop2	Laptop3	Laptop4	Laptop5	Laptop6
1	4572441859	4991040350	6250824972	5066123098	4209558769	3348434613
2	2314236509	2687831850	3173183580	2735508010	2165006731	1821799779
4	1217033100	1279695475	1992610220	1680698909	1370837658	1211187931
8	689617692	742187427	1452965087	939522946	1060941914	1234086582
16	499438032	481031573	1487359785	522585137	884125449	1275989687
32	536416350	525633126	1465158280	566259775	868553805	1220860877
64	515050602	508655173	1445463006	506929752	874676401	1219027990

Threads	Laptop1	Laptop2	Laptop3	Laptop4	Laptop5	Laptop6
1	1.235795	1.2635545	1.89418939	1.164626	1.336368	1.455841
2	0.625469	0.6804638	0.96157078	0.628852	0.687304	0.792087
4	0.328928	0.3239735	0.60382128	0.386368	0.435187	0.526603
8	0.186383	0.1878956	0.44029245	0.215982	0.336807	0.536559
16	0.134983	0.1217801	0.45071509	0.120135	0.280675	0.554778
32	0.144977	0.1330717	0.44398736	0.130175	0.275731	0.530809
64	0.139203	0.1287735	0.43801909	0.116536	0.277675	0. 529786

Rezultate pentru înmulțirea matricelor

Threads	Laptop1	Laptop2	Laptop3	Laptop4	Laptop5	Laptop6
1	2318511823	2297622954	2733501609	2052636710	1905285141	1621441650
2	1119724175	1037780093	1332444233	1127287642	1031859817	692774532
4	568444194	569461763	729524994	653140269	542361358	373094463
8	313536492	369510240	436302068	381248611	376419686	436918056
16	223060890	243870342	482302233	259695115	379780837	442655838
32	227815926	263890806	464552968	191624789	377726479	417628942
64	229162932	246559303	452702307	195079450	360424326	1621441650

Threads	Laptop1	Laptop2	Laptop3	Laptop4	Laptop5	Laptop6
1	0.626625	0.5816767	0.82833382	0.471871	0.604852	0.704975
2	0.302628	0.2627291	0.40377098	0.259147	0.327575	0.301206
4	0.153634	0.1441675	0.22106818	0.150147	0.172178	0.162215
8	0.08474	0.0935469	0.13221275	0.087643	0.119498	0.189964
16	0.060287	0.0617393	0.14615219	0.0597	0.120565	0.189964
32	0.061572	0.0668078	0.14077363	0.044052	0.119913	0.181578
64	0.061936	0.0624201	0.13718252	0.044846	0.11442	0.174224

Rezultate pentru transformate Fourier rapidă

Threads	Laptop1	Laptop2	Laptop3	Laptop4	Laptop5	Laptop6
1	3355104618	3361785178	3079094473	2834101591	3715801559	2196781881
2	1695961381	1708597003	1643609564	1606827645	1833390020	1189009746
4	920803877	901874069	1026382709	1086578918	1049279112	670333941
8	512386356	593003703	784588066	661659338	850042957	682344229
16	429791010	402521984	820609069	509384808	839902709	674887080
32	473507844	420841527	782381562	399741372	844056353	667306230
64	448223886	419324394	788506201	448173310	795019106	688823298

Threads	Laptop1	Laptop2	Laptop3	Laptop4	Laptop5	Laptop6
1	0.906785	0.8510849	0.93305893	0.651518	1.17962	0.516961
2	0.458368	0.4325562	0.4980635	0.369386	0.582029	0.29145
4	0.248866	0.2283225	0.31102506	0.249788	0.333104	0.296671
8	0.138483	0.1501275	0.23775396	0.152106	0.269855	0.293429
16	0.11616	0.1019043	0.24866941	0.1171	0.266636	0.290133
32	0.127975	0.1065422	0.23708532	0.091895	0.267954	0.299488
64	0.121142	0.1061581	0.23894127	0.103028	0.252387	0.516961

8.3. Metodologia de notare și sistemul de punctaj

În vederea obținerii unei evaluări comparative a procesoarelor testate, a fost definit un sistem de notare bazat pe numărul total de cicluri raportate de cele trei benchmark-uri implementate. Numărul de cicluri reprezintă este un indicator robust al performanței arhitecturale brute, fiind direct și independent de frecvența raportată a procesorului.

Pentru fiecare categorie de test, s-a stabilit un interval între valoarea minimă (cel mai performant sistem) și valoarea maximă (cel mai slab sistem). Pe baza acestui interval, fiecare procesor a primit o notă de la 1 la 10, conform poziționării sale relative. Scorul final al unui procesor reprezintă media aritmetică a notelor obținute la cele trei benchmark-uri.

Ponderea testelor este: 30% 8t, 30% 16t, 20% 1t, 20% restul.

Monte Calro	
Nota	Cicluri maxime
1	2,844,486,723
2	2,585,897,021
3	2,370,405,603
4	2,188,066,710
5	2,031,776,231
6	1,896,324,482
7	1,777,804,202
8	1,673,227,484
9	1,580,270,402
10	1,497,098,276

Matrix Multiplication	
Nota	Cicluri maxime
1	1,295,841,683
2	1,178,037,893
3	1,079,868,069
4	996,801,294
5	925,601,202
6	863,894,455
7	809,901,052
8	762,259,813
9	719,912,046
10	682,021,938

FFT	
Nota	Cicluri maxime
1	2,027,649,340
2	1,843,317,582
3	1,689,707,784
4	1,559,730,262
5	1,448,320,957
6	1,351,766,227
7	1,267,280,838
8	1,192,734,906
9	1,126,471,856
10	1,067,183,864

9. Concluzii

9.1. Probleme întâmpinate

Una dintre primele probleme întâmpinate a fost crearea proiectului într-un mod corect, astfel încât să pot utilizat limbajul de asamblare inline. Acest lucru a impus o anumită versiune de WinForms, de setări de compilare (x86) și configurare a Visual Studio-ului.

De asemenea, una dintre principalele probleme a fost utilizarea instrucțiunii CPUID, în special din cauza diferențelor de implementare între producători, dar și între procesoarele aceluiași producător. Fiecare producător expune informațiile într-un mod ușor diferit, dar suficient ca informațiile returnate să nu mai fie corecte. Găsirea registrelor corecte pentru fiecare tip de arhitectură pe care am testat-o și am observat că are probleme la detecția hardware a reprezentat și ea la rândul ei o problemă, deoarece adesea nu se găsesc cu ușurință documentații pentru această instrucțiune, fiind nevoie de documentațiile procesoarelor în sine.

O altă problemă de care m-am lovit a fost să mă asigur că toate benchmark-urile rulează într-un număr relativ apropiat de cicluri de ceas. Acest aspect este esențial deoarece ajută la o mai bună vizualizare a graficelor rezultate. Dacă existau diferențe foarte în numărul de cicluri de ceas, atunci ar fi fost mult mai dificilă analiza performanței, deoarece unele dintre grafice s-ar fi asemănat cu o linie în comparație cu celelalte. Pentru o obține valorile care să ne aducă la un număr de cicluri destul de apropiat a fost nevoie de mai multe încercări cu diferite dimensiuni pentru testele implementate.

Nu în ultimul rând, standardizarea pentru testare a fost în sinea ei o problemă. Ca rezultatele obținute de benchmark să fie relevante a fost nevoie să fie asigurată o stare specifică pentru sistemul pe care a fost testat, asta însemnând să existe cât mai putințe programe care rulau în paralel cu benchmark-ul. Toate acestea au fost esențiale deoarece un sistem care poate obține o notă de 10 în condiții perfecte, poate să ia o notă de 7 dacă există prea multe alte procese în paralel.

9.2. Posibile îmbunătățiri

Totodată, pe viitor, proiectul poate fi extins și rafinat prin:

- Stabilizarea și standardizarea modului de testare: setarea afinității thread-urilor (thread pinning), dezactivarea pe cât posibil a migrației între nuclee și introducerea unor rulări de pregătire înainte de măsurarea efectivă;
- Extinderea setului de benchmark-uri cu sarcini care testează alte componente ale procesorului (acces memorie, lățime de bandă, etc.);
- Îmbunătățirea exportului prin includerea automată a informațiilor despre CPU în fișierul exportat, precum și salvarea graficelor (PNG) sau a unui format structurat (JSON) pentru procesare automată;
- Parametrizare mai flexibilă (dimensiunea matricelor, numărul de puncte), astfel încât testele să poată fi ajustate în funcție de sistem și de timpul disponibil;
- Raport automat: generarea unui sumar final cu note per benchmark și un scor final agregat, pentru comparații rapide între sisteme.

9.3. Concluzie

Așadar, în cadrul acestui proiect a fost dezvoltată o aplicație de benchmarking pentru evaluarea performanței procesoarelor multicore, bazată pe sarcini reprezentative. Aplicația integrează o bibliotecă DLL implementată în C# cu o interfață grafică dezvoltată în C#, demonstrând o abordare eficientă, care îmbină funcționalitatea codului nativ cu ușurința de utilizare a părții grafice. Aplicația îndeplinește obiectivele propuse, punând la dispoziție un instrument perfect pentru analizarea și compararea performanței procesoarelor, dar și o bază solidă pentru extinderi și optimizări ulterioare.

Bibliografie

[1] Bouckaert, Stefan &Vanhie-Van Gerwen, Jono &Moerman, Ingrid & Phillips, Stephen&Wilander, Jerker. (2011). BONFIRE: benchmarking computers and computer networks. [accesat Oct. 2025] https://eprints.soton.ac.uk/374081/1/374081.pdf

[2] SPEC's Benchmarks and Tools [accesat Oct. 2025], https://www.spec.org/benchmarks.html

[3] G. M. Amdahl, Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities, 1967 [accesat Oct. 2025], https://www3.cs.stonybrook.edu/~rezaul/Spring-2019/CSE613/reading/Amdahl-1967.pdf

[4] Date and time library [accesat Oct. 2025], https://en.cppreference.com/w/cpp/chrono.html

[5] Intel, Using the RDTSC Instruction for Performance Monitoring [accesat Oct. 2025], https://www.ccsl.carleton.ca/~jamuir/rdtscpm1.pdf

[6] QueryPerformanceCounter function (profileapi.h) [accesat Oct. 2025], https://learn.microsoft.com/en-us/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter
[7] CPUID [accesat Oct. 2025], https://en.wikipedia.org/wiki/CPUID
[8] The MonteCarlo Method, Nicholas Metropolis & S.Ulam, Journal of the American Statistical Association, Vol. 44, No, 247 (Sep 1949) [accesat Oct. 2025], https://web.williams.edu/Mathematics/sjmiller/public_html/105Sp10/handouts/MetropolisUlam_TheMonteCarloMethod.pdf
[9] Matrix multiplication via arithmetic progression, Don Coppersmith, Shmuel Winograd[accesat Oct. 2025], https://www.sciencedirect.com/science/article/pii/S0747717108800132
[10] The Design and Implementation of FFTW3, Matteo Frigo and Steven G. Johnson [accesat Oct. 2025], https://math.mit.edu/~stevenj/papers/FrigoJo05.pdf
[11] CPUID – CPU Identification, Arhitectura sistemelor de calcul, UBB, Alexandru Vancea [accesat Oct. 2025], https://www.cs.ubbcluj.ro/~vancea/asc/practic/html/CPUID.html
[12] CPUID Specification, Publication#25481, Revision 2.28, Issue Date April 2008 [accesat Oct. 2025], https://kib.kiev.ua/x86docs/AMD/AMD-CPUID-Spec/25481-r2.28.pdf?utm_source=chatgpt.comv \
[13] Recommendation for Random Number Generation Using Deterministic Random Bit Generators, Elaine Barker, John Kelsey [accesat Oct. 2025] https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-90Ar1.pdf
[14] An Algorith for the Machine Calculation of Complex Fourier Series, By James W. Cooley and John W. Tukey [accesat Nov. 2025] https://web.stanford.edu/class/cme324/classics/cooley-tukey.pdf
[15] Inside the FFT BlackBox: Serial and Parallel Fast Fourier Transform Algorithms (Computational mathematics), Eleanor Chu & Alan George, chap. 5 - 9
[16] Intel® 64 and IA-32 Architectures Software Developer's Manual, [accesat Ian. 2026] https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html